动态视频眼动数据至静态全局图映射方案

一、 方案概述与核心挑战

业务场景：用户观看一段浏览《清明上河图》等高分辨率静态长图的动态视频。眼动仪记录了用户相对于**视频播放窗口（动态帧）的注视点坐标（X, Y）。需求是将这些视频坐标，精准反算、投射到原始高清静态长图**的绝对坐标系中，以便进行全局的热力图或轨迹分析。

核心挑战：

1. **视角不断变化**：视频的每一帧只展示了全图的一个局部，且可能伴随平移、缩放（Zoom in/out）。
2. **时空同步**：眼动数据的时间戳必须与视频的帧序列严格对齐。
3. **计算性能**：逐帧进行全图特征匹配计算量巨大，需要工程级别的优化。

二、 核心解算原理：单应性矩阵（Homography）

解决此问题的唯一标准路径是**图像特征匹配与单应性变换**。

视频的某一帧画面，本质上是对全景图在特定视角下的一次“采样”。如果能在视频帧和全景图之间找到足够多的“相同特征点”（例如画卷中的某座桥、某个人物），就可以计算出一个 3×3 的变换矩阵（单应性矩阵 H）。

通过这个矩阵，视频帧上的任意一点 (x_{vid}, y_{vid})，都可以通过纯数学计算，直接映射到全局图上的 (X_{static}, Y_{static})。

三、 系统工作流设计

整个数据处理流建议在 **Python 后端** 离线或异步完成，不建议在前端浏览器中进行。

步骤 1：数据预处理与时序对齐

- **输入**：`video.mp4`（如 30fps）和 `raw_gaze.json`（包含 `timestamp, x, y`）。
- **处理**：将眼动数据的时间戳 t 转换为视频的帧索引 $Frame_Index$。
  - 公式：$Frame_Index = \text{Round}(t \times FPS)$
  - 确保每一条眼动数据都绑定到了特定的一帧图像上。

步骤 2：图像特征提取与匹配

使用 OpenCV 提取当前视频帧（局部）和静态底图（全局）的特征点。

- **特征算法选择**：
  - **针对《清明上河图》等艺术画作**：画面线条细密、可能存在重复纹理，**强烈建议使用 SIFT 算法** 或深度学习匹配器（如 SuperGlue / LightGlue）。传统的 ORB 算法速度虽快，但在这种复杂线稿上容易产生大量误匹配。
- **匹配过程**：
  - 使用 FLANN 匹配器对两张图的特征描述子进行 KNN 匹配。
  - 使用 Lowe's Ratio Test 剔除劣质匹配点。

步骤 3：单应性矩阵解算

- 获取到一对一的优质匹配点坐标后，使用 OpenCV 的 `cv2.findHomography` 配合 **RANSAC**（随机抽样一致性）算法。
- RANSAC 会自动忽略匹配错误的离群点，计算出最稳健的 3×3 变换矩阵 H。

步骤 4：坐标仿射与映射

拿到矩阵 H 后，将当前帧的眼动坐标 (x, y) 转换为齐次坐标 $(x, y, 1)$，并与矩阵相乘：

$$\begin{bmatrix} X' \\ Y' \\ W \end{bmatrix} = H \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

将结果除以缩放因子 W 进行归一化，得到静态全图上的绝对坐标：

- **目标 X**：$X_{static} = X'/W$
- **目标 Y**：$Y_{static} = Y'/W$

四、 关键代码逻辑演示 (Python + OpenCV)

```
import cv2
import numpy as np

def map_gaze_to_static(video_frame, static_map, gaze_x_vid, gaze_y_vid):
    # 1. 初始化 SIFT 探测器
    sift = cv2.SIFT_create()

    # 2. 提取特征点和描述子
    kp1, des1 = sift.detectAndCompute(video_frame, None)
    kp2, des2 = sift.detectAndCompute(static_map, None)

    # 3. FLANN 特征匹配
    index_params = dict(algorithm=1, trees=5)
    search_params = dict(checks=50)
    flann = cv2.FlannBasedMatcher(index_params, search_params)
    matches = flann.knnMatch(des1, des2, k=2)

    # 4. 筛选优秀匹配点 (Lowe's ratio test)
    good_matches = []
    for m, n in matches:
        if m.distance < 0.7 * n.distance:
            good_matches.append(m)

    if len(good_matches) > 10:
        # 获取匹配点坐标
        src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).reshap
        dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).reshap

        # 5. 计算单应性矩阵 H (使用 RANSAC)
```

```
        H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

        # 6. 进行注视点坐标映射
        gaze_point = np.array([[[gaze_x_vid, gaze_y_vid]]], dtype=np.float32)
        mapped_point = cv2.perspectiveTransform(gaze_point, H)

        return mapped_point[0][0][0], mapped_point[0][0][1]
    else:
        return None, None # 匹配失败，帧模糊或特征不足
```

五、 工程化落地与性能优化 (防坑指南)

如果视频长达 1 分钟（1800 帧），逐帧进行 SIFT 计算会导致处理时间极其漫长。必须引入工程优化：

1. **关键帧抽取 + 矩阵复用（极大提速）**：
   - 视频的连续相邻帧位移极小。不需要每帧都算矩阵。
   - **策略**：每隔 15 帧（约 0.5 秒）抽取一个“关键帧”计算一次矩阵 H。
   - **插值**：这 15 帧中间的眼动点，可以直接复用关键帧的矩阵，或者通过前后的矩阵进行线性插值，运算速度可提升 10 倍以上。
2. **光流法 (Optical Flow) 追踪**：
   - 更高级的优化是：只在第 1 帧计算单应性矩阵。后续帧通过 OpenCV 的 `calcOpticalFlowPyrLK` 追踪画面的平移量 (dx, dy)，直接在初始矩阵上叠加位移。
3. **分块搜索 (ROI 优化)**：
   - 视频帧只占《清明上河图》很小的一块。在第二帧匹配时，不要用整张《清明上河图》去匹配，而是根据第一帧的位置，在全图上裁剪出一个稍微大一点的 ROI (感兴趣区域) 进行匹配，可以指数级减少计算量。
4. **处理运动模糊**：
   - 视频在快速平移时会产生运动模糊，导致特征提取失败（单应性矩阵计算失败）。代码中必须加入异常处理：如果当前帧匹配失败，则继承上一成功帧的矩阵参数，并标记数据置信度降低。

六、 最终输出与前端整合

Python 脚本跑完后，抛弃视频，直接导出一份干净的 JSON 文件交由前端渲染：

```
{
  "image_ref": "qingming_shanghe_tu_highres.jpg",
  "resolution": {"width": 24000, "height": 1000},
  "gaze_data": [
    {"timestamp": 0.033, "x": 1250, "y": 450, "duration": 150},
    {"timestamp": 0.066, "x": 1255, "y": 452, "duration": 300}
    // ... 映射后的绝对坐标
  ]
```

```
}
```

前端（React/Vue）读取高清大图和这个 JSON，利用 Canvas/WebGL 技术绘制平滑的轨迹图和热力图即可。此时前端彻底解耦，无需关心视频同步和复杂的坐标转换逻辑。